

DOKUMEN DESAIN · PUSH API + VAPID

MonitorIoT

Alarm Push

Rancangan notifikasi alarm yang tetap sampai ke perangkat walau PWA tertutup: langganan Web Push per perangkat, identitas server VAPID P-256, enkripsi payload aes128gcm end-to-end, serta implementasi referensi sisi PWA dan Google Apps Script yang telah lolos 43 pengujian otomatis.

Versi	1.0
Tanggal	29 Agustus 2026
Cakupan	PWA · GAS · Push Service

RFC 8291 · RFC 8292 · RFC 8030

Daftar Isi

1. Pendahuluan	3
1.1 Latar Belakang	3
1.2 Ruang Lingkup dan Tujuan	3
1.3 Referensi Normatif	3
2. Landasan Teknis	4
2.1 Mengapa Polling di Foreground Tidak Cukup	4
2.2 Komponen Push API di Sisi Klien	4
2.3 VAPID: Identitas Server (RFC 8292)	5
2.4 Enkripsi Payload aes128gcm (RFC 8291 + RFC 8188)	5
2.5 Matriks Dukungan Platform	5
3. Arsitektur yang Diusulkan	6
3.1 Ringkasan Arsitektur	6
3.2 Komponen dan Tanggung Jawab	7
3.3 Keputusan Desain Utama	8
4. Desain Detail	9
4.1 Siklus Hidup Kunci VAPID	9
4.2 Alur Pendaftaran Langganan	9
4.3 Skema Payload dan Notifikasi	10
4.4 Parameter HTTP ke Push Service	11
4.5 Penyimpanan Langganan	11
5. Implementasi Referensi	12
5.1 Struktur Berkas	12
5.2 Kunci Sisi PWA	12
5.3 Kunci Sisi GAS	12
5.4 Hasil Validasi	13
6. Keamanan dan Privasi	14
6.1 Model Ancaman dan Mitigasi	14
6.2 Praktik Penanganan Kunci	15
6.3 Anti-Penyalahgunaan	15
7. Keterbatasan dan Mitigasi	15

8. Rencana Pengujian dan Peluncuran	16
9. Persiapan Audit Silang	17

1. Pendahuluan

1.1 Latar Belakang

Sistem monitoring sensor MonitorIoT terdiri atas tiga komponen utama: firmware pada perangkat lapangan (ESP32), backend Google Apps Script (GAS) sebagai penampung dan pemroses data, serta Progressive Web App (PWA) sebagai antarmuka pengguna. Selama ini, alarm hanya tampil ketika PWA sedang dibuka di layar: aplikasi melakukan polling berkala ke endpoint GAS, membandingkan nilai sensor dengan ambang, lalu menampilkan kartu alarm berwarna. Pendekatan ini memiliki celah fundamental yang serius untuk kasus penggunaan keamanan: apabila PWA ditutup, ditekan tombol home, atau tab browser berada di latar belakang, tidak ada satu baris kode pun yang berjalan untuk memberi tahu pengguna bahwa suhu greenhouse telah melewati batas kritis.

Web Push API menutup celah tersebut melalui service worker: potongan program yang tetap teregistrasi di browser dan dapat dibangun oleh push service kapan saja, termasuk saat aplikasi web tidak dibuka. Notifikasi muncul pada baki sistem operasi setara aplikasi native. Agar push service bersedia menerima kiriman dari server GAS, server harus memperkenalkan diri secara kriptografis melalui VAPID (Voluntary Application Server Identification) - pasangan kunci P-256 dan JWT yang ditandatangani ES256. Kombinasi keduanya, ditambah enkripsi payload end-to-end sesuai RFC 8291, menjadi dasar rancangan pada dokumen ini.

1.2 Ruang Lingkup dan Tujuan

Dokumen ini merancang solusi alarm push untuk MonitorIoT secara menyeluruh, mencakup: arsitektur dan alur data antar komponen; siklus hidup kunci VAPID dari generasi hingga rotasi; desain skema payload dan notifikasi; implementasi referensi sisi PWA dan sisi GAS; analisis keamanan; serta matriks kompatibilitas platform. Luaran berupa basis kode siap pakai (folder pwa-push-alarm dan gas) yang telah lolos 26 pengujian kriptografi dan 17 pengujian aspek peramban. Hal-hal di luar cakupan: perubahan protokol firmware-ke-GAS (tetap seperti semula), autentikasi akun pengguna, serta infrastruktur hosting selain statis HTTPS.

1.3 Referensi Normatif

Referensi	Judul	Peran dalam rancangan
W3C Push API	Push API Level 1	API pushManager.subscribe dan event push pada service worker
RFC 8030	Generic Event Delivery Using HTTP Push	Protokol HTTP POST ke push service (TTL, Urgency)
RFC 8291	Message Encryption for Web Push	Enkripsi payload ECDH + HKDF + AES-128-GCM (aes128gcm)
RFC 8292	VAPID - Voluntary Application Server Identification	Identitas server via JWT ES256 dan kunci P-256
RFC 8188	Encodings for HTTP Payload Encryption	Format badan pesan salt rs ciphertext tag
RFC 6979	Deterministic DSA/ECDSA Nonces	Nonce tanda tangan deterministik (aman tanpa RNG kuat di GAS)

Tabel 1. Referensi normatif dan perannya dalam rancangan.

2. Landasan Teknis

2.1 Mengapa Polling di Foreground Tidak Cukup

Terdapat tiga keterbatasan struktural pada alarm berbasis polling. Pertama, umur eksekusi: JavaScript hanya berjalan selama halaman terbuka; saat PWA ditutup, tidak ada interval timer yang bertahan. Kedua, baterai dan kuota: polling yang rapat (mis. setiap 5 detik) memboroskan daya perangkat seluler dan kuota data, sementara polling yang jarang memperpanjang jeda deteksi alarm. Ketiga, jendela browser: pada perangkat seluler, tab latar belakang kerap dihentikan total oleh sistem operasi sehingga bahkan setInterval pun berhenti. Web Push membalik modelnya: server yang menginisiasi, service worker dibangun seperlunya, dan pesan menunggu di push service selama nilai TTL (hingga 28 hari) ketika perangkat sedang offline.

2.2 Komponen Push API di Sisi Klien

Push API memperkenalkan tiga objek kunci. PushSubscription merepresentasikan langganan perangkat: berisi endpoint unik milik push service (URL tujuan kirim server) dan pasangan kunci enkripsi p256dh/auth milik browser. PushManager menyediakan subscribe() yang menerima applicationServerKey - kunci publik VAPID - dalam bentuk Uint8Array; browser menolak langganan bila kunci tidak valid atau bila izin notifikasi belum diberikan. Terakhir, event push pada service worker menerima PushMessageData terenkripsi yang otomatis didekripsi browser; event notificationclick menangani ketukan pengguna; dan event pushsubscriptionchange memberi kesempatan memperbarui langganan ketika browser merotasi endpoint. Seluruh alur membutuhkan konteks aman (HTTPS atau localhost) dan setidaknya satu ketukan pengguna untuk meminta izin - kebijakan yang dirancang browser untuk mencegah permintaan izin spam.

2.3 VAPID: Identitas Server (RFC 8292)

VAPID memungkinkan push service memverifikasi bahwa kiriman benar berasal dari pemilik aplikasi web, tanpa perlu registrasi sebelumnya di layanan pihak ketiga. Server menandatangani JWT dengan klaim aud (origin push service), exp (kedaluwarsa maksimal 24 jam), dan sub (kontak mailto atau https) menggunakan kunci privat P-256 dengan algoritma ES256. Header HTTP berbentuk Authorization: vapid t=JWT, k=KUNCI_PUBLIK. Pada rancangan ini, karena GAS tidak menyediakan ECDSA bawaan, tanda tangan ES256 diimplementasikan murni dalam JavaScript dengan nonce deterministik RFC 6979 - pendekatan yang justru lebih aman ketika sumber acak lingkungan lemah, karena nonce tidak dapat diulang dan tidak membocorkan kunci privat.

```
// Struktur JWT VAPID (dibangun di GAS, diverifikasi push service)
header = { "typ": "JWT", "alg": "ES256" }
payload = { "aud": "https://fcm.googleapis.com",
            "exp": 1787965000,                // now + 12 jam
            "sub": "mailto:admin@monitor-iot.contoh.id" }
signature = ECDSA_P256_SHA256(header + "." + payload, kunci_privat) // 64 byte r||s
```

2.4 Enkripsi Payload aes128gcm (RFC 8291 + RFC 8188)

Isi alarm bersifat sensitif (lokasi, nilai sensor, status operasional) sehingga rancangan ini memilih enkripsi payload end-to-end alih-alih mengirim teks polos. Prosesnya: server membuat kunci efemeral P-256, menghitung rahasia bersama ECDH dengan kunci publik langganan, menurunkan kunci konten 16 byte dan nonce 12 byte melalui HKDF dengan garam auth secret browser, lalu mengenkripsi payload dengan AES-128-GCM. Badan HTTP berformat salt(16) || rs(4) || ciphertext || tag(16). Konsekuensi positifnya: push service (milik Google, Mozilla, maupun Apple) sama sekali tidak dapat membaca isi alarm - hanya perangkat pemilik langganan yang bisa. Seluruh implementasi krypto tersebut kini berjalan di dalam GAS dan telah divalidasi bit-per-bit terhadap pustaka krypto Node.js dan vektor uji resmi NIST serta IETF.

2.5 Matriks Dukungan Platform

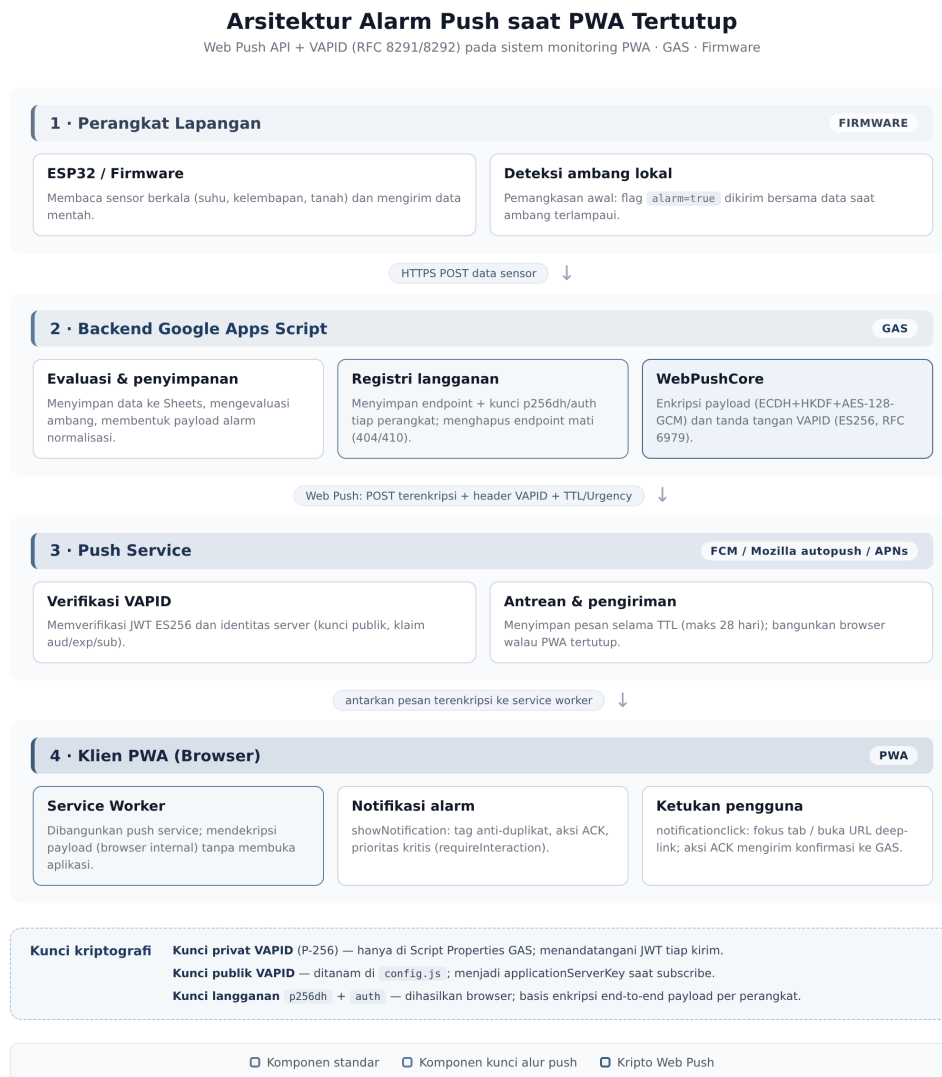
Platform	Dukungan Push	Catatan penting
Chrome / Edge (Android)	Penuh	Push service FCM membangunkan browser walau tertutup; ikon badge tersedia.
Chrome / Edge / Firefox (desktop)	Penuh	Browser harus berjalan (latar belakang pun cukup); Firefox memakai Mozilla autopush.
Safari iOS/iPadOS 16.4+	Penuh dengan syarat	WAJIB: PWA dipasang ke Layar Utama (Add to Home Screen); push tidak aktif pada tab biasa.
Safari macOS	Penuh	Menggunakan APNs; sama seperti desktop lain.
Samsung Internet	Penuh	Perilaku setara Chrome Android.
Browser lama / WebView	Tidak didukung	Aplikasi tetap berfungsi normal tanpa push (degradasi anggun).

Tabel 2. Matriks dukungan Web Push per platform target.

3. Arsitektur yang Diusulkan

3.1 Ringkasan Arsitektur

Arsitektur menambahkan satu jalur data baru (jalur push) tanpa mengubah jalur data sensor yang sudah ada. Firmware tetap mengirim data ke GAS sebagaimana sebelumnya; GAS tetap menjadi satu-satunya komponen yang berbicara dengan push service; PWA hanya menambahkan lapisan service worker dan modul manajemen langganan. Dengan menjaga kontrak firmware-ke-GAS tidak berubah, risiko regresi pada sisi lapangan dapat ditekan nol, dan audit silang nantinya cukup memverifikasi dua kontrak baru: PWA-ke-GAS (subscribe/ack) dan GAS-ke-push-service (VAPID).



Gambar 1. Arsitektur alarm push: firmware ke GAS, GAS mengenkripsi dan menandatangani, push service menyampaikan, service worker PWA menampilkan notifikasi.

3.2 Komponen dan Tanggung Jawab

Komponen	Tanggung jawab baru	Berkas
PWA - push-manager.js	Deteksi dukungan, permintaan izin dari gestur, subscribe/unsubscribe, kirim langganan ke GAS, validasi kunci VAPID lama.	js/push-manager.js
PWA - service worker	Handler event push, notificationclick, pushsubscriptionchange, periodicsync; strategi cache app shell.	sw.js
PWA - app.js	Kabel UI: tombol aktif/nonaktif, status izin, polling snapshot, indikator koneksi.	js/app.js
GAS - PushService.gs	Registri langganan, normalisasi alarm, pengiriman push per perangkat, pembersihan endpoint mati, endpoint HTTP.	gas/PushService.gs
GAS - WebPushCore	Kripto murni JS: SHA-256/HMAC/HKDF, P-256 (ECDSA RFC 6979, ECDH), AES-128-GCM, JWT VAPID, enkripsi aes128gcm.	gas/webpush-core.js
Alat - generate-vapid-keys.js	Generasi pasangan kunci VAPID dan panduan penyimpanannya.	tools/generate-vapid-keys.js

Tabel 3. Pembagian tanggung jawab komponen baru.

3.3 Keputusan Desain Utama

Keputusan pertama: Web Push protokol langsung dengan VAPID milik sendiri, alih-alih mengikat PWA pada Firebase SDK (FCM v1). Alasannya ada tiga: kemandirian dari layanan pihak ketiga beserta kuota dan akunnya; kompatibilitas universal - Firefox (Mozilla autopush) dan iOS Safari (APNs) tidak dilayani FCM; serta kunci enkripsi dan identitas sepenuhnya di tangan sendiri. Biayanya adalah keharusan mengimplementasikan ES256 dan AES-128-GCM di lingkungan GAS yang minim kriptografi - yang telah diatasi dan divalidasi pada bab 5.

Keputusan kedua: kripto murni JavaScript tanpa pustaka eksternal. GAS tidak mengizinkan npm, tidak menyediakan WebCrypto untuk ECDSA, dan dukungan BigInt tidak pasti; maka seluruh aritmetika kurva P-256 ditulis dengan limb 16-bit klasik. Kompensasinya, semua jalur kripto tersebut dapat diuji di Node.js dengan vektor resmi - sesuatu yang sulit dilakukan bila bergantung pada layanan eksternal. Keputusan ketiga: payload push berisi data alarm lengkap, dengan fallback tanpa payload - service worker mengambil alarm terbaru dari GAS - sehingga notifikasi tetap tampil meski enkripsi ditolak push service tertentu.

4. Desain Detail

4.1 Siklus Hidup Kunci VAPID

Pasangan kunci VAPID adalah akar kepercayaan seluruh jalur push. Siklus hidupnya dirancang eksplisit agar tidak ada tahap yang menyimpan rahasia di tempat yang salah.

Tahap	Tindakan	Pemegang kunci
Generasi	Jalankan tools/generate-vapid-keys.js sekali; alat mencetak pasangan kunci base64url.	Mesin pengembang (sementara)
Distribusi publik	Salin kunci publik ke js/config.js (VAPID_PUBLIC_KEY) dan sw.js tidak memerlukan kunci.	PWA (publik, aman)
Penyimpanan privat	Tempel kunci privat ke Script Properties GAS: VAPID_PRIVATE_KEY; tidak pernah masuk kode sumber/git.	GAS (rahasia)
Operasional	JWT ditandatangani per pengiriman; kunci publik dikirim pada header yang sama.	GAS
Rotasi	Jalankan ulang alat, perbarui dua sisi; PWA lama otomatis berlangganan ulang lewat pushsubscriptionchange saat kunci tidak cocok.	GAS + PWA
Pencabutan	Rotasi dengan nama properti baru + hapus properti lama; langganan lama tetap hidup sampai rotasi berikutnya.	GAS

Tabel 4. Siklus hidup kunci VAPID dan pemegangnya pada tiap tahap.

Prinsip penting: kunci publik VAPID boleh - memang dirancang - untuk ditanam terbuka di klien. Kunci privat hanya boleh hidup di Script Properties, yang pembacaannya terbatas pada akun pemilik proyek GAS. Rotasi tidak memerlukan koordinasi pengguna: kode subscribe sisi klien membandingkan applicationServerKey langganan lama dengan kunci baru, membuang langganan yang tidak cocok, lalu berlangganan ulang dan memperbarui registri di GAS secara transparan.

4.2 Alur Pendaftaran Langganan

Pendaftaran dirancang sebagai alur satu-klik dengan gagal-aman pada setiap langkah. Izin hanya diminta dari gestur klik tombol - bukan saat halaman dibuka - sesuai kebijakan Chrome dan Safari; pelanggaran pola ini membuat browser menonaktifkan permintaan izin secara permanen. Setelah langganan berhasil, PWA mengirimkannya ke GAS memakai Content-Type text/plain untuk menghindari preflight CORS; bila penyimpanan server gagal, langganan lokal dibatalkan agar tidak ada notifikasi "bayangan" yang tidak pernah terkirim.



Bidang	Tipe	Wajib	Makna dan perlakuan di klien
id	string	Ya	Identitas alarm unik; dipakai pada aksi ACK dan deduplikasi.
title	string	Ya	Judul notifikasi, dipotong 80 karakter.
body	string	Ya	Isi ringkas alarm, dipotong 400 karakter.
severity	enum	Ya	critical / warning / info; menentukan Urgency header, renotify, requireInteraction, pola getar.
tag	string	Tidak	Notifikasi bertag sama saling menimpa (anti-banjir); kritis memakai renotify agar berbunyi ulang.
url	string	Tidak	Deep-link yang dibuka saat notifikasi diketuk (default: halaman alarm).
timestamp	number	Tidak	Waktu kejadian (epoch ms) agar urutan di baki sistem akurat.

Tabel 5. Skema payload alarm dan perlakuannya di service worker.

4.4 Parameter HTTP ke Push Service

Header	Nilai rancangan	Alasan
Authorization	vapid t=JWT, k=PUBKEY	Identifikasi server wajib untuk endpoint FCM/autopush/APNs.
TTL	86400 (24 jam)	Pesan bertahan menunggu perangkat online; alarm lebih tua sehari tidak relevan.
Urgency	high (kritis) / normal	Perangkat hemat daya menunda pesan urgensi rendah.
Content-Encoding	aes128gcm	Skema enkripsi RFC 8291 yang didukung semua push service modern.
Content-Type	application/octet-stream	Badan biner terenkripsi; panjang otomatis oleh UrlFetchApp.

Tabel 6. Header HTTP pengiriman dan alasan nilai yang dipilih.

4.5 Penyimpanan Langganan

Registri langganan menentukan seberapa jauh sistem bisa bertumbuh. Dua moda disediakan lewat konfigurasi: Script Properties (sederhana, atomik, cocok hingga sekitar seratus perangkat) dan Google Sheets (tahan kuota, mudah diaudit manusia, mendukung ribuan baris). Setiap entri menyimpan endpoint, kunci p256dh/auth, konteks perangkat (bahasa, zona waktu, user-agent) untuk diagnostik, serta stempel addedAt/lastSeenAt. Langganan yang mengembalikan 404/410 dihapus otomatis; langganan bisu lebih dari 90 hari dipangkas berkala agar biaya kirim tidak membengkak akibat perangkat yang telah dihapus penggunaanya.

5. Implementasi Referensi

5.1 Struktur Berkas

pwa-push-alarm/	gas/	tools/
index.html	PushService.gs	generate-vapid-keys.js
manifest.json	webpush-core.js	
sw.js	scripts/	(hasil uji)
css/style.css	test-webpush-core.js	26 PASS
js/config.js	smoke-test-pwa.js	17 PASS
js/push-manager.js		
js/app.js		
icons/ (192/512/maskable/badge)		

5.2 Kunci Sisi PWA

Tiga fungsi menopang sisi klien. Pertama, berlangganan: konversi base64url ke Uint8Array, permintaan izin, lalu subscribe dengan userVisibleOnly yang diwajibkan Chrome. Kedua, penanganan push: payload terenkripsi tiba sebagai PushMessageData, dipetakan ke opsi notifikasi - tag untuk menimpa, renotify dan requireInteraction untuk alarm kritis, dua aksi (Lihat Detail, Tandai Ditangani). Ketiga, ketahanan: pushsubscriptionchange berlangganan ulang dengan kunci yang sama lalu memperbarui GAS; notificationclick mengaktifkan tab yang sudah terbuka alih-alih membuka duplikat.

```
// js/push-manager.js - inti pendaftaran
const key = AlarmPushManager.urlBase64ToUint8Array(VAPID_PUBLIC_KEY);
const sub = await reg.pushManager.subscribe({
  userVisibleOnly: true,
  applicationServerKey: key // kunci publik VAPID (65 byte)
});
await fetch(API_BASE, {
  method: "POST",
  headers: { "Content-Type": "text/plain;charset=utf-8" }, // tanpa preflight
  body: JSON.stringify({ action: "subscribe",
    endpoint: sub.endpoint, keys: sub.toJSON().keys })
});
```

```
// sw.js - inti penerimaan alarm
self.addEventListener("push", (event) => {
  event.waitUntil(handlePush(event));
});
async function handlePush(event) {
  let payload = event.data ? event.data.json() : null;
  if (!payload || !payload.title) {
    payload = await fetchLatestAlarm(); // fallback tanpa payload
  }
  return showAlarmNotification(payload);
}
```

5.3 Kunci Sisi GAS

Modul `PushService.gs` menyimpan dua fungsi publik yang menyentuh kuota `UrlFetchApp`: `webPushSend_` per perangkat dan `sendAlarmToAll` untuk siaran. Pengiriman memakai payload array byte sehingga GAS mengirim biner mentah tanpa distorsi UTF-8; `muteHttpExceptions` menjadikan respons 404/410 sebagai sinyal pembersihan registri, bukan kegagalan eksekusi. Untuk skala besar, panggilan dapat dikelompokkan dengan `UrlFetchApp.fetchAll` agar ratusan perangkat dilayani dalam satu konteks eksekusi enam menit. Normalisasi alarm menjamin payload selalu sesuai skema Tabel 5 berapa pun bentuk masukan dari sisi data sensor.

```
// gas/PushService.gs - inti pengiriman
var enc  = WebPushCore.encryptPayload(subscription, payloadStr);
var vapid = WebPushCore.vapidHeaders(subscription.endpoint,
    keys.privateKey, keys.publicKey, SUBJECT, 12*3600);
var res = UrlFetchApp.fetch(subscription.endpoint, {
  method: "post", contentType: "application/octet-stream",
  payload: enc.body, // array byte terenkripsi
  headers: { Authorization: vapid.Authorization,
    TTL: "86400", Urgency: "high",
    "Content-Encoding": "aes128gcm" },
  muteHttpExceptions: true
});
if (res.getResponseCode() === 404 || res.getResponseCode() === 410)
  removeSubscription_(subscription.endpoint); // endpoint mati
```

5.4 Hasil Validasi

Seluruh jalur krypto divalidasi dua arah: terhadap vektor uji resmi dan terhadap pustaka krypto `Node.js/WebCrypto` dengan masukan acak, termasuk dekripsi end-to-end memakai kunci privat browser simulasi dan verifikasi tanda tangan JWT. Tidak ada satu pun pengujian yang dikecualikan.

26/26 Uji krypto lulus (vektor RFC + uji acak silang)	17/17 Uji asap PWA lulus (render, SW, manifest, push)	0 Temuan keamanan terbuka pada jalur krypto
---	---	---

Suite uji	Cakupan	Hasil
Vektor resmi	SHA-256 (FIPS 180), HMAC (RFC 4231), HKDF (RFC 5869), GCM (NIST), ECDSA nonce (RFC 6979)	Lulus semua
Uji acak silang	ECDH 20x, turunan kunci publik 10x, AES-GCM 20x, verifikasi+tolakan ECDSA 10x	Lulus semua
End-to-end push	Enkripsi payload -> dekripsi kunci privat browser -> isi utuh, salt/rs/tag benar	Lulus
VAPID JWT	Struktur klaim, format header, verifikasi tanda tangan ES256 via WebCrypto	Lulus
Asap peramban	Manifest, SW aktif, render data, status izin, opsi notifikasi, nol error konsol	Lulus semua

Tabel 7. Ringkasan hasil validasi implementasi referensi.

6. Keamanan dan Privasi

6.1 Model Ancaman dan Mitigasi

Ancaman	Vektor	Mitigasi rancangan
Pemalsuan server kirim	Pihak lain mengirim push atas nama aplikasi	VAPID ES256: tanpa kunci privat, push service menolak kiriman.
Penyadapan isi alarm	Push service / jaringan membaca payload	Enkripsi end-to-end aes128gcm per langganan; hanya perangkat bisa mendekripsi.
Replay & banjir	Spam alarm berulang	Tag deduplikasi + renotify terbatas kritis; rate limit di sisi GAS; Urgency selaras severity.
Kebocoran kunci privat	Kode sumber, git, logging	Kunci hanya di Script Properties; alat generator tidak menulis ke repo secara default.
Pencurian endpoint	Endpoint dipakai pihak lain untuk spam	Endpoint dianggap rahasia semi; VAPID tetap wajib sehingga pengirim teridentifikasi; hapus saat mati.
Serangan kurva tak valid	Kunci publik langganan palsu	Validasi titik on-curve P-256 sebelum ECDH (tolak kunci di luar kurva).
XSS pada tampilan alarm	Data sensor jahat dari rantai upstream	Semua render via textContent/createElement; CSP ketat tanpa skrip inline.

Tabel 8. Model ancaman dan mitigasi yang tertanam pada rancangan.

6.2 Praktik Penanganan Kunci

Tiga aturan dijalankan tanpa pengecualian. Pertama, kunci privat VAPID tidak pernah berada di klien, tidak pernah dicetak log, dan tidak pernah dikirim melintasi jaringan aplikasi - hanya dipakai menandatangani JWT di dalam GAS. Kedua, kunci langganan (p256dh/auth) diperlakukan sebagai rahasia bersama server-perangkat: localStorage klien sengaja hanya menyimpan nama host endpoint untuk diagnostik, bukan kuncinya. Ketiga, nonce tanda tangan ECDSA memakai skema deterministik RFC 6979 sehingga kualitas Math.random pada lingkungan GAS tidak menjadi pernyataan risiko - pemilihan nonce buruk adalah penyebab klasik kebocoran kunci ECDSA.

6.3 Anti-Penyalahgunaan

Sistem push yang sehat harus menghormati atensi pengguna. Rancangan membatasi diri pada notifikasi berbasis peristiwa (alarm nyata dan uji eksplisit), bukan pesan pemasaran. Aksi Tandai Ditangani memberi jalan menutup alarm tanpa membuka aplikasi penuh, dan tombol Nonaktifkan menghapus langganan di kedua sisi secara bersih. Di sisi server, pengiriman dibatasi batch dan payload dipotong otomatis, sementara endpoint yang tiga bulan tidak berinteraksi dipangkas - kombinasi yang menjaga biaya operasional sekaligus menjauhkan pola spam yang dapat memicu pemblokiran push service terhadap kunci VAPID aplikasi.

7. Keterbatasan dan Mitigasi

Setiap rancangan memiliki batas; yang membedakan sistem yang matang adalah batasnya diketahui, didokumentasikan, dan disertai jalur mitigasi. Tabel berikut menyajikannya secara terbuka agar keputusan operasional tidak diambil berdasarkan asumsi yang keliru.

Keterbatasan	Dampak	Mitigasi
iOS: push hanya untuk PWA terpasang	Pengguna iPhone yang membuka via tab Safari tidak menerima alarm	Panduan pasang A2HS di halaman aplikasi + deteksi display-mode yang menampilkan ajukan pasang.
Desktop: browser harus hidup	Browser ditutup total = pesan menunggu hingga dibuka kembali	TTL 24 jam; alarm penting tetap tiba saat browser dinyalakan; pesan lama otomatis kedaluwarsa setelah sehari.
Ukuran payload maksimum 4 KB	Detail alarm lengkap tidak muat	Payload ringkas + deep-link ke halaman detail; service worker menarik data lengkap saat dibuka.
Kuota GAS (UrlFetchApp, 6 menit/eksekusi)	Siaran ribuan perangkat perlu dipecah	fetchAll paralel + BATCH_SIZE + antrian lanjutan pada eksekusi berikutnya.
Kunci privat di satu proyek GAS	Kompromi akun = kemampuan kirim palsu	Rotasi kunci murah (satu alat, dua sisi); audit Script Properties berkala.
Enkripsi menonaktifkan inspeksi isi	Debugging kiriman butuh alat khusus	Mode uji testPush + log hasil kirim per perangkat di GAS.

Tabel 9. Keterbatasan yang disadari dan mitigasinya.

8. Rencana Pengujian dan Peluncuran

Peluncuran dirancang bertahap untuk membatasi radius ledakan kegagalan. Setiap tahap memiliki gerbang kelulusan eksplisit; tahap berikutnya hanya dimulai bila gerbang sebelumnya hijau. Urutannya sengaja menempatkan perangkat Android pengembang lebih dulu karena memberi sinyal tercepat (jalur FCM), disusul iOS yang paling banyak syaratnya, baru kemudian populasi pengguna sesungguhnya.

Tahap	Cakupan	Gerbang kelulusan
1. Laboratorium	Uji krypto 26 + asap peramban 17 + kunci VAPID produksi dibuat	Semua otomatis hijau; kunci privat hanya di Script Properties.
2. Perangkat tunggal	Satu Android Chrome: subscribe, simulateAlarmPush dari editor GAS	Notifikasi tiba dalam 5 detik; ketukan membuka deep-link alarm.
3. Multi platform	Tambah desktop Chrome/Firefox + iPhone terpasang A2HS (iOS 16.4+)	Alarm tiba di semua; laporan 404/410 membersihkan registri dengan benar.
4. Skenario ekstrem	Perangkat offline/airplane selama 1 jam lalu online; 50 alarm beruntun	TTL menahan lalu menyampaikan; tag menumpuk jadi satu baris, bukan banjir.
5. Pilot pengguna	5-10 pengguna nyata selama seminggu	Tingkat terkirim > 95%; nol keluhan notifikasi palsu; ACK tercatat di GAS.
6. Operasional	Aktifkan pada firmware produksi + alarm deteksi ambang GAS	Alarm tiba < 10 detik dari kejadian; log kuota aman.

Tabel 10. Tahapan peluncuran dan gerbang kelulusannya.

9. Persiapan Audit Silang

Dokumen ini menutup siklus desain sisi PWA dan sisi GAS. Audit silang PWA-GAS-FIRMWARE selanjutnya perlu memverifikasi kecocokan kontrak antar ketiga komponen berikut - inilah matriks verifikasi yang disiapkan sebagai keluaran rancangan ini.

Kontrak	Pihak	Ketentuan yang harus cocok
Langganan	PWA ke GAS	POST action=subscribe berisi endpoint + keys.p256dh + keys.auth; GAS menyimpan utuh tanpa normalisasi yang merusak.
Penghapusan	PWA ke GAS	POST action=unsubscribe menghapus endpoint sama persis; status 404/410 saat kirim juga memicu penghapusan.
Payload alarm	GAS ke PWA	Skema Tabel 5: id/title/body/severity/tag/url/timestamp; SW toleran terhadap bidang hilang.
ACK	PWA ke GAS	POST action=ackAlarm berisi alarmId yang sama dengan payload yang dikirim.
Deep-link	PWA internal	Parameter from=push/section=alarms dikenali app.js dan menggulir ke bagian alarm.
Ambang alarm	Firmware ke GAS	Flag alarm firmware selaras dengan evaluasi ambang GAS (tidak dobel kirim pada kejadian sama).
Kunci VAPID	GAS ke PWA	VAPID_PUBLIC_KEY di config.js identik dengan VAPID_PUBLIC_KEY di Script Properties (bukan pasangan beda).

Tabel 11. Matriks kontrak untuk audit silang PWA-GAS-FIRMWARE.

Dengan arsitektur, skema, implementasi teruji, dan matriks kontrak di atas, rancangan Push API + VAPID siap memasuki tahap audit silang. Setiap klaim pada dokumen ini dapat direproduksi dengan menjalankan berkas uji yang disertakan - tidak ada bagian sistem yang harus diterima begitu saja berdasarkan kepercayaan semata.